PFP IV: Native Verification and the Custody of Finite Evidence

Jeremy H. Carroll

2026-10-02 · Version 0.1.0 · Systems methods paper · Author-review revision

Code baseline: 52cb5457f6c485ba548cfd292f6012a941ed22cf

Abstract

The Proposal Fidelity Protocol (PFP), formerly the Symbol Game, separates untrusted proposal generation from checking a declared finite property. This paper describes the concrete native verification architecture at source commit 52cb5457f6c485ba548cfd292f6012a941ed22cf. Its contribution is an account of the interfaces and evidence boundaries required to preserve that separation across Python objects, canonical encodings, a Rust process, exact solver replay, and portable outcome records. A worked four-vertex fixture illustrates the complete path: the host reconstructs a carrier, recomputes packet and request identities, replays an exact solve, compares the proposed normal form, and returns exact laboratory evidence. The same result refuses admission because three conformance laws remain unchecked and its attestation is unsigned. We distinguish this restricted host route from the crate's broader runtime dispatcher and additive foundation library. We also distinguish the repository's tagged encoding and sorted JSON profiles from RFC 8785. The system offers finite integrity and replay checks under explicit assumptions; it does not establish producer authenticity, arbitrary Python/Rust equivalence, a reproducible source-to-binary build, or authority to deploy a result.

1. The problem and the contribution

A generated answer may be syntactically acceptable, mathematically valid on one finite carrier, and unsuitable for operational admission. Collapsing those judgments into a single success flag loses essential information. The risk grows when a Python caller serializes an object, invokes a native solver, and later presents only an unqualified "verified" label.

PFP addresses this engineering problem by retaining distinct artifacts. A proposal identifies what should be evaluated. The host reconstructs what it can evaluate. An execution outcome records which checks occurred, which exact values were obtained, and which obligations remain. A policy decision determines whether the result may cross a consequential boundary. The examined host's execution records confer no operational authorization.

This paper contributes a source-pinned interface inventory, a complete fixture trace, and a threat model for interpreting the emitted commitments. It also identifies deliberately unavailable paths and uneven canonicalization contracts. It is a systems and methods report. Its bounded test cases are evidence about their exercised paths; a passing test suite would not prove every reachable program behavior.

The scope is the Rust `omega_symbol_game` crate, its protocol, the Python native seam wrapper, and the named transport and regression sources below. The repository that holds those files is private. This deposit includes an immutable copy of the audited source at the code baseline, audited_source/pfp-iv-native-source-52cb5457.tar. The GitHub links name that revision. The archive is the copy a reader can inspect. The four-vertex calculation used here is restated in Section 6 and in the companion study guide. PFP I is a separate preprint in this series and is not required to check

this fixture. The present paper explains how an implementation records a finite calculation without enlarging its meaning.

2. Related work and design assumptions

Proof-carrying code studies a producer supplying evidence that a consumer checks against a safety policy. The relevant architectural precedent is explicit consumer-side checking. PFP's laboratory records have a different scope and are not a proof that arbitrary generated code satisfies a safety policy (Necula, 1997).

Cryptographic comparison requires agreement about the bytes being compared. RFC 8785 specifies JSON canonicalization rules covering primitive serialization and deterministic property sorting. It also requires preservation of string data rather than Unicode normalization. The repository has multiple encoding profiles, so conformance cannot be inferred from a function named `canonical_json` (Rundgren, Jordan, and Erdtman, 2020).

Our execution assumptions are explicit: the declared carrier is finite; the arithmetic profile is known; the checker and native binary selected by the launcher are the intended ones; the operating environment executes those bytes; and any expected digest anchor is retained independently of the artifact being checked. Invalidating one assumption requires a narrower interpretation of the result. Exact arithmetic alone does not discharge the other assumptions.

3. Three interfaces, three different capabilities

The crate exposes three distinguishable surfaces.

Surface	Entry	Evidence boundary
Restricted prime host	Default executable operation, `evaluate_request_bytes`	One same-carrier candidate, exact replay, explicit admission result
Runtime dispatcher	`invoke`, `--describe-runtime`, `dispatch_runtime_request`	Per-action payload and outcome contracts
Additive foundation	Rust library `foundation`	Finite exhaustion, extension, packet, and composition witnesses

The executable source also offers `--describe`, `--derive-carrier-manifest`, and `classify-board`. `--describe` reports the restricted host profile; `--describe-runtime` reports adapter names derived from the runtime action table. Descriptions are useful discovery artifacts, but advertising an action does not establish that every mathematical input or every external use is supported.

The dispatcher includes functional evaluation and replay, exact matrix operations, cochain checks, code-change actions, and other named adapters. A runtime action called `prime_admit` must be interpreted under that action's own contract. Its existence does not remove the restricted host's refused admission result. Likewise, a finite conservative-extension witness in the foundation library does not wire a carrier-changing lift into the default host protocol.

Python's omega_engine_symbol_game.ffi.RustSymbolGameHost delegates process location and invocation to the operating-system adapter. The supported `OMEGA_SYMBOL_GAME_HOST` environment

override belongs to that seam. Convenience methods retain the full railway outcome or typed refusal. Equivalent names in Python and Rust are insufficient to establish functional parity: parity needs a declared common input domain, output relation, and differential evidence.

A separate tutorial adapter, omega_engine_providers.adapters.native_functional.compute_graph_phi1, requires OMEGA_FUNCTIONAL_GRAPH_BIN and OMEGA_FUNCTIONAL_GRAPH_SHA256. It validates the executable digest and exact graph inputs, then parses a narrowly shaped native result. This is a separate binary-selection contract. Configuring one environment variable family does not configure the other.

4. Packet identity and canonicalization

The principal Python transport type is `PacketEnvelopeV2`. Its identity includes the task and packet family, carrier, initial state, payload, constraints, protected boundaries, terminal target, evidence ceiling, oracle policy, allowed transforms, budget, framing, and source/parent bindings. The payload is frozen through the domain canonical representation. Parsing the transport object does not create a verified solve receipt or a continuation permit.

Three serialization distinctions must survive documentation:

Encoding surface	Actual contract at the audited revision
Domain canonical algebra	Tagged values, including exact integers, fractions, and supported compound types
`PacketCodec` helper	NFC normalization followed by tagged domain encoding
Restricted host profile	`omega-sorted-json-no-float-v1` for its outer request structure

The domain canonical module explicitly encodes type information. For example, an integer and a string containing the same digits remain distinguishable. It validates canonical compound encodings on reconstruction. The `PacketCodec` helper explicitly says its bytes are not RFC 8785. Its `verify` method checks payload digest and authority; it does not validate every field sealed into its `envelope_sha256`. It must not be substituted for the richer protocol verifier when claiming complete envelope custody.

The host's outer no-float profile and the Python tagged payload algebra operate at different layers. The host tests include canonical tagged float and fraction images, while theorem-bearing arithmetic uses exact rational values. Accepting a typed representation in a payload does not license floating-point approximation in an exact solver input.

Canonicalization establishes reproducible representation under a profile. Hashing establishes an integrity comparison under an anchor. Semantic equivalence requires an explicit projection or checker. None of these steps, alone, authenticates who produced the record.

5. Restricted host execution

The host implementation admits the following narrow successful path. The request uses omega.oesos.rust-prime-host-request.v1, carries exactly one candidate, and binds the host registry and build-stamped source revision. The arithmetic domain is rational and the acceptance mode

is must_equal_replayed_normal_form-v1. This restricted request wire accepts weights and before/candidate coefficients only as minimal signed 128-bit integer strings; it cannot express every rational input. The solver computes with `BigRational` internally. The broader `invoke` dispatcher has its own input contract, which must be checked separately.

First, bounded intake rejects malformed, duplicate, unknown, or oversized request data. The host recomputes request and packet identities rather than trusting caller-supplied values. It validates the carrier declaration, derives its manifest, and compares the version and carrier hash to the supplied anchor. Before and candidate vectors must match the edge dimension.

The host next performs the same-carrier identity checks and executes the exact solve. The solve receipt remains inside the Rust process and is replayed there. The candidate is accepted for this laboratory execution only if it exactly equals the replayed rational normal form. A caller cannot obtain that result by changing a candidate digest consistently.

Subsequent checks interpret exact final masses under an integral `u64` policy; they do not round nonintegral values to fit the policy. Conformance records preserve unchecked laws. The supported route consumes one internal permit for an exact write and another for terminalization. Those permit objects are not serialized; the outcome carries only digest bindings for them.

Finally, an outcome carries rational summaries, typed refusal reasons, commitments, and an outcome root. Even a fully executed route may return refused admission. Exported bytes are a summary of the process evidence; retaining the internal receipt across solve and replay prevents a Python caller from manufacturing the particular Rust evidence object used in that sequence.

6. Worked example: exact repair with refused admission

The canonical host integration fixture uses vertices 0, 1, 2, 3, unit-weight oriented edges

`(0,1), (1,2), (2,0), (1,3), (3,0)`

and triangular faces `(0,1,2)`, `(0,1,3)`. Its before vector is `(3,2,2,1,1)` and its candidate vector is `(0,0,0,0,0)`. The fixture constructs the manifest, computes the packet ID, obtains the host description, and computes the request ID using exported helpers.

This carrier matters: deleting a face changes what closure means, even if the edge list remains identical. Consequently the manifest commits to faces as well as vertices and edges. A graph-only certificate cannot silently become a certificate for this two-dimensional carrier.

The test specifies the following expected observations:

Observation	Expected value
Execution status	EXECUTED_REPLAYED_EXACT_LAB_EVIDENCE
Replayed candidate relation	Exact match
Intermediate gauge summaries	Phi1 = 7; Gamma2 = 12
Final summaries	Phi1 = 0; Gamma2 = 0
Conformance	`Partial`; three unchecked laws
Internal consumed letters	`E,T`; two budget units
Attestation	`unsigned_lab`
Admission	`Refused`

Observation	Expected value
Admission reasons	PRIME000_CONFORMANCE_PARTIAL, `UNSIGNED_LAB_ATTESTATION`

The intermediate values can be checked by hand: the before vector circulates $3 + 2 + 2 = 7$ around face `(0,1,2)` and $3 + 1 + 1 = 5$ around face `(0,1,3)`, so Gamma2, the unweighted L1 norm of these face values, is $7 + 5 = 12$, while Phi1, the unit-weight L1 distance from the before vector to the edge differences of vertex potentials, is 7, attained by the residual `(5,2,0,0,0)` and matched from below by the unit circulation around `(0,1,2)`. The two faces span both independent cycles of this graph, so the replayed normal form is the zero vector and the final values are 0 and 0.

These are assertions of the test at the pinned source. Section 9 separates that source text from the retained historical native run. They illustrate how numerical success and an unmet admission obligation coexist. A consumer that displays only the final zeros would erase the most important qualification.

The same suite supplies counterexamples. Changing a carrier while retaining its old manifest anchor is refused, even when outer request hashes are recomputed. Changing a candidate and recomputing its surrounding identity still fails if it is not the replayed normal form. Editing a field of a returned outcome breaks its root check; because the root is unkeyed, recomputing it would restore only internal consistency, not create a fresh native replay, and the outcome remains unsigned laboratory evidence with refused admission. Each case targets a distinct attempted shortcut through custody.

The novice interpretation is direct: the laboratory completed a specified calculation, but the result lacks the additional evidence needed for admission. “Refused” here is not a statement that arithmetic failed.

7. Resource boundaries and unavailable workflows

The restricted host accepts at most 1,048,576 request bytes. It bounds declared vertices, edges, and faces, and the sum of vertices and faces, at 4,096 each. Its preallocation fence also bounds four dense exact matrices (d0, d1, and two augmented systems) at 1,048,576 cells each; dimension products are checked before allocation. The supported successful route needs two prime writes; the declared budget range is 2–64.

The timeout field is validated metadata (1 to 300,000 milliseconds), not an in-process watchdog. A caller requiring a wall-clock kill boundary must enforce it around the process and record that enforcement. Exact rational operations can still be expensive within dimension limits; the paper claims no general throughput or polynomial bound from these fences.

Runtime `invoke` has its own 65,536-byte input ceiling. The intake reader consumes at most the limit plus one witness byte, which permits an oversized request to be rejected without reading an unbounded stream. These limits belong to different interfaces and should be reported separately.

The legacy Python `run-turn`, `run-bout`, and `run` CLI routes are currently required to return `RUST_FUNCTIONAL_REQUIRED` before creating output. Boundary regressions document that retirement. The supported offline catalog commands and native `gate11-self` path provide more precise entry points. In these engine paths, missing native configuration is an unavailable calculation, not a zero defect or a silently substituted Python estimate. The companion governor in this package follows

the same rule when a native executable is configured, and without one it computes the minimum by exact enumeration (Section 10).

For the graph tutorial, the historical operator fixture supplies a small hand check: claims (1,1,1) with penalties (1,2,2) have quotient minimum 3 and an attaining projected cochain (-2,1,1). The series' shared foundation example uses penalties (1,2,3): its minimum is also 3, while its zero-potential cost is 6 rather than this fixture's 5. A chosen section's residual and the minimum over all sections are different quantities. Comparing two Python wrappers that call the same function is an API compatibility check, not independent solver agreement. The weighted-cycle note, docs/completion_supplement.md, gives the argument and describes the companion code in this package.

8. Threat model and evidence claims

The adversary may control proposal text, serialized fields, stale outcomes, and files offered as witnesses. The reviewed protections target malformed intake, carrier drift, wrong candidate values, stale commitments, and unsupported requests. The trusted computing base includes the selected native executable, its dependencies, exact arithmetic implementation, protocol code, and execution environment.

An unkeyed outcome root cannot authenticate the producer. An attacker able to replace both a record and its expected anchor can recompute both. The source revision is a build stamp; the host description does not itself supply a trusted executable digest or signed reproducible-build statement. Several source bindings are committed names or digests whose underlying source bytes are not independently reconstructed by the default host.

The interpretation must therefore remain conditional. An independently pinned executable hash identifies executed bytes; it does not prove how those bytes were built. A fresh replay supplies evidence about the declared input and checker; it does not show that the carrier represents an external physical system faithfully. Operational authorization requires its own policy and trusted provenance.

This package also retains six small Lean modules from its former manuscript in `lean4/`: incidence-sign and signed column-sum lemmas, identity-path and quotient lemmas, rigidity of one oriented two-point relation, the weighted null-space inequalities and unique landing for one directed 3-cycle, a list-general directional-derivative bound, and 60 elementary set and point-set topology lemmas. They build with the pinned toolchain, and `lean4/axiom_report.txt` lists the axioms used by each of the 86 theorems. None of them formalizes the native host, the packet protocol, or a receipt field. No end-to-end refinement theorem connecting every Python object, native action, receipt field, and Lean statement is asserted here.

9. Reproducibility and verification plan

To repeat the native checks, extract audited_source/pfp-iv-native-source-52cb5457.tar, record the actual toolchain, and run only the relevant targets from the extracted root. The principal native commands are:

```
cargo test -j 1 --locked --manifest-path omega_engine/crates_rs/Cargo.toml -p
    omega_symbol_game --test host_contract
cargo test -j 1 --locked --manifest-path omega_engine/crates_rs/Cargo.toml -p
    omega_symbol_game --test foundation_contract
```

```
cargo test -j 1 --locked --manifest-path omega_engine/crates_rs/Cargo.toml -p
    omega_symbol_game --test invoke_contract
cargo test -j 1 --locked --manifest-path omega_engine/crates_rs/Cargo.toml -p
    omega_symbol_game --test native_cli_intake
```

`host_contract` checks the worked path and hostile variations. `foundation_contract` includes a cross-language golden packet vector, incomplete exhaustion rejection, detector preservation, and composition/replay refusals. Those shared vectors establish the declared examples, not parity on every input. A historical author-run report records the four native targets on 2026-09-28 EDT (2026-09-29 UTC), using Rust 1.98.0 on Windows (x86_64-pc-windows-msvc). It reports `host_contract` 12 passed, `foundation_contract` 11, `invoke_contract` 32 and `native_cli_intake` 9, with no failures. The run commit is 1c7092c3db2ab2fa65999cf2555aff7733b97b99. The `omega_symbol_game` crate tree at that commit and at the code baseline is 75d77aa5e1f30dfc102f00c8f7cb6159418dec63. This is retained historical execution evidence. The 2026-10-02 package-local receipt does not record a fresh run of these native targets.

The report is in this deposit at docs/historical_native_run/report_second_revision_2026-09-28.md. The two JSON files beside it record the package review pipeline from the same window. They do not name these four Cargo targets. Raw Cargo standard output was not retained. The public source copy is audited_source/pfp-iv-native-source-52cb5457.tar, with member digests in audited_source/SOURCE_MEMBERS.sha256. Reproduction uses `--locked` and that pinned tree, including its workspace and `Cargo.lock`. A later checkout is a different tree.

For Python, collect and run `test_packet_contracts.py`, test_exact_functional_port.py, test_fun ctional_boundary.py, and test_ffi_seam_isolation.py under omega_engine_symbol_game/tests/. The exact-functional suite contains scripted refusal cases and native cases. Its native cases can skip when the executable is missing; a receipt must retain that distinction. A scripted host tests adapter handling, not native arithmetic.

A complete receipt records commands, exits, interpreter and Rust versions, source digests, binary digest, environment-selected host, and separate passed, failed, and skipped counts. Descriptive source expectations in this paper must not be rewritten into measured results until that receipt exists. Provider calls and model downloads are unnecessary for these checks.

The package itself needs no repository checkout. From the package directory, run one test file at a time with `py -3.12 -m pytest code/tests/<file>`, build the Lean modules with `lake build` in `lean4/`, and replay the axiom census with `py -3.12 tools/replay_lean_axiom_census.py --replay` .. The package tests check its legacy companion modules and the weighted-cycle note. They run the governor's exact Python enumeration unless a native graph executable is configured, so they are not evidence for the native host described above.

10. Limitations and practical use

PFP's useful guarantee is local: a consumer can inspect which finite calculation was requested, which checks were replayed, which identities were compared, and why admission remained unavailable. The inspected host does not provide carrier-changing transport, arbitrary licensed lifts, multiple candidates, or a fully authenticated deployment path.

The architecture also contains legacy helpers with weaker verification contracts. Documentation must name the exact API rather than infer its properties from shared terminology. Uniform RFC 8785 compliance, global Rust/Python parity, and complete source-to-binary custody remain outside

the demonstrated scope. The companion code in this package is one example. Its governor computes the graph minimum by exact Python enumeration when no native executable is configured. Until this revision it also substituted that enumeration whenever a configured native call raised, including when the native adapter refused an input; it now lets every native refusal or error propagate and refuses floating-point or Boolean edge weights on both paths.

This separation enables honest use. A developer can reproduce a finite experiment and retain its refusal without claiming that all proof obligations or operational decisions have been settled. Future work should target one missing boundary at a time, with a falsifying example and a new source-bound receipt.

This paper assigns no authorization and changes no jurisdiction. It carries no operational authority or institutional standing.

Package verification

The package verifier (`verify_deposit.py`) checks the manifest and the two-way file census, and `tools/rebuild_release.py --check` checks the release archive against the manifests. The current package-local record is docs/release_validation_2026_10_02_review_repairs.json. It lists each command, tool version, exit status, and tested scope for the checks run with this repair. It does not record a Cargo run or a Lean build. docs/release_validation_2026_10_01.json and docs/release_validation_2026_10_02.json keep their original dates and scope.

References

- Necula, G. C. (1997). Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*, pp. 106–119. ACM. DOI: 10.1145/263699.263712.
- Rundgren, A., Jordan, B., and Erdtman, S. (2020). JSON Canonicalization Scheme (JCS). RFC 8785 (Informational), RFC Editor, June 2020. DOI: 10.17487/RFC8785.

Implementation links throughout the paper identify the exact audited source revision. Section 9 identifies the historical native run and its limits. The public inspection copy is audited_source/pfp-iv-native-source-52cb5457.tar.